From the Line Filter to the Anisotropic Stencil Filter

A Step-by-Step Mathematical Walkthrough

J.C. Vaught

1 The Big Picture (Before Any Math)

The whole point of this document is to answer one question: how did we go from the Line Filter (LF), which is slow and memory-hungry, to the Anisotropic Stencil Filter (ASF), which is fast and lean, without changing the answer at all?

The short version is this. The Line Filter does a bunch of steps (gather pixels, build a matrix, solve a system, repeat along a line, weight, sum). Every single one of those steps is a linear operation on the original pixel values. When you chain a bunch of linear operations together, the result is still just one big linear operation. That means the entire LF pipeline, no matter how complicated it looks, is secretly equivalent to "multiply each pixel in a neighborhood by some weight and add them up." The ASF simply precomputes those weights ahead of time so the GPU only has to do the easy part: gather, multiply, sum. One operation instead of many. Same answer. Much faster.

The rest of this document walks through every step of the math that makes this work, slowly and with lots of explanation.

2 Step 1: What Are We Even Computing?

2.1 The Goal

We have a grayscale image, which is really just a grid of numbers. Each number is a brightness value. We want to find edges, the places where the brightness changes sharply.

To find edges, we need to estimate how fast the brightness is changing at every pixel. In calculus terms, we want the derivative of the image intensity. The bigger the derivative, the stronger the edge.

2.2 The Problem with Simple Approaches

The most obvious way to estimate a derivative is to look at two neighboring pixels and take the difference. This is essentially what Sobel and Prewitt filters do (with some slight averaging). The problem is that if the image is noisy, those two pixels might have random junk in their values, and the difference will be meaningless.

Our approach uses way more pixels (say, 100 instead of 9) and fits a smooth polynomial surface through all of them. The derivative of that smooth surface is our edge estimate. More pixels means more noise averaging, and the polynomial fit means we are not blurring edges the way Gaussian smoothing does.

2.3 Why Direction Matters

An edge has a direction. A vertical edge has a strong horizontal derivative and a weak vertical one. A diagonal edge has strong derivatives in neither the horizontal nor vertical direction individually, but a strong derivative perpendicular to the edge.

Rather than computing derivatives in just two directions (horizontal and vertical) and combining them, we try many candidate directions (say, 18 angles evenly spaced around the circle) and pick the one that gives the biggest response. This is what "anisotropic" means in the name: the filter looks different in different directions.

3 Step 2: The Point Filter (One Pixel, One Direction)

This is the foundation of everything. Before we worry about lines or stencils, we need to understand how to estimate a derivative at a single pixel in a single direction.

3.1 Setting Up Local Coordinates

Pick a target pixel at position (X_0, Y_0) in the image. Pick a candidate edge direction θ_k (one of our 18 angles, for example). We set up a local coordinate system centered on our pixel, rotated so that x points along the normal to the edge (the direction we want the derivative in) and y points along the edge itself.

For any neighbor pixel at (X_i, Y_i), its local coordinates are:

$$\begin{aligned} x_i &= (X_i - X_0)\cos\theta_k + (Y_i - Y_0)\sin\theta_k \\ y_i &= -(X_i - X_0)\sin\theta_k + (Y_i - Y_0)\cos\theta_k \end{aligned} \tag{1}$$

This is just a coordinate rotation. We are re-expressing each neighbor's position relative to our target pixel, in a coordinate system aligned with the candidate edge direction.

To give a concrete example, suppose our target pixel is at $(50, 50)$ and we are testing $\theta_k = 0°$ (a vertical edge). Then the rotation is trivial: $x_i = X_i - 50$ and $y_i = Y_i - 50$. For $\theta_k = 45°$, the axes are rotated 45 degrees, so the x and y coordinates become mixtures of the horizontal and vertical offsets.

3.2 Fitting a Polynomial Surface

Now we model the image brightness near our target pixel as a polynomial in these local coordinates. Using a Taylor expansion of order d (we typically use $d = 4$):

$$f(x_i, y_i) \approx \underbrace{f^0}_{\text{constant}} + \underbrace{f^0_x x_i}_{\text{linear in } x} + \underbrace{f^0_y y_i}_{\text{linear in } y} + \underbrace{\frac{f^0_{xx}}{2} x_i^2}_{\text{quadratic}} + \cdots \tag{2}$$

There are $M = (d+1)\frac{d+2}{2}$ unknown coefficients in this expansion. For $d = 4$, that is $M = 15$ unknowns.

What are these unknowns? They are the partial derivatives of the image at our target pixel: the value f^0, the first derivatives f^0_x and f^0_y, the second derivatives f^0_{xx}, f^0_{yy}, f^0_{xy}, and so on up to fourth order. The one we care about most is f^0_x, the derivative in the normal direction, because that is our edge strength.

3.3 Gathering Neighbors

We select N_p neighbor pixels around our target. These are chosen as the N_p closest integer-coordinate pixels, forming an approximately circular patch. For $N_p = 100$, this circle has a radius of about 6 pixels.

Each neighbor gives us one equation: we know its local coordinates (x_i, y_i) from Equation 1, and we know its brightness $f(X_i, Y_i)$ from the image. Plugging into Equation 2 gives us one equation relating the 15 unknowns to one known brightness value.

With $N_p = 100$ equations and 15 unknowns, we have a heavily overdetermined system. This is good because it means the solution averages out noise.

3.4 The Linear System

Stack all N_p equations into a matrix equation:

$$\boldsymbol{A}_{\theta_k} \boldsymbol{z} = \boldsymbol{b} \tag{3}$$

Here is what each piece is.

$\boldsymbol{A}_{\theta_k}$ is an $N_p \times M$ matrix (e.g., 100×15). Each row corresponds to one neighbor pixel. The entries in that row are the monomial terms evaluated at that neighbor's local coordinates. For example, for a 2nd-order fit ($d = 2,\ M = 6$), row i would be $\left[1, x_i, y_i, \frac{x_i^2}{2}, \frac{y_i^2}{2}, x_i y_i\right]$. For $d = 4$ there are 15 such terms.

$\boldsymbol{z}$ is the $M \times 1$ vector of unknowns: $\left[f^0, f_x^0, f_y^0, \frac{f_{xx}^0}{2}, ...\right]$.

$\boldsymbol{b}$ is the $N_p \times 1$ vector of observed brightness values: $\left[f(X_1, Y_1), f(X_2, Y_2), ..., f\left(X_{N_p}, Y_{N_p}\right)\right]$.

3.5 Solving It: The Pseudoinverse

Since we have more equations than unknowns, there is no exact solution. We find the least-squares best fit:

$$\hat{\boldsymbol{z}} = \left(\boldsymbol{A}_{\theta_k}^\top \boldsymbol{A}_{\theta_k}\right)^{-1} \boldsymbol{A}_{\theta_k}^\top \boldsymbol{b} \tag{4}$$

We define the pseudoinverse matrix:

$$\boldsymbol{P}_{\theta_k} = \left(\boldsymbol{A}_{\theta_k}^\top \boldsymbol{A}_{\theta_k}\right)^{-1} \boldsymbol{A}_{\theta_k}^\top \tag{5}$$

This is an $M \times N_p$ matrix. It transforms a vector of N_p brightness values into the M fitted coefficients. So $\hat{\boldsymbol{z}} = \boldsymbol{P}_{\theta_k} \boldsymbol{b}$.

3.6 Extracting the Derivative We Want

We only need $\hat{f}_x^0$, the normal derivative, which is the second entry of $\hat{\boldsymbol{z}}$ (index 1 if we start counting from 0). That means:

$$\hat{f}_x^{(k)} = \boldsymbol{p}_{\text{fx}}^{(k)} \cdot \boldsymbol{b} \tag{6}$$

where $\boldsymbol{p}_{\text{fx}}^{(k)}$ is row 1 of $\boldsymbol{P}_{\theta_k}$, a vector of N_p numbers.

This is a key insight. The estimated derivative is just a dot product between a fixed weight vector $\boldsymbol{p}_{\text{fx}}^{(k)}$ and the raw pixel brightness values $\boldsymbol{b}$. No matter how fancy the polynomial fitting sounds, in the end it is just "multiply each neighbor's brightness by a weight and add up."

The weights $\boldsymbol{p}_{\text{fx}}^{(k)}$ depend on the geometry (where the neighbors are and what angle we are testing) but not on the image. They can be computed once in advance.

3.7 Picking the Best Direction

We repeat this for all N_s candidate angles $\theta_k = k \cdot 2\frac{\pi}{N_s}$ and pick the direction with the strongest response:

$$k^* = \arg\max_k |\hat{f}_x^{(k)}| \tag{7}$$

The gradient magnitude is $|\hat{f}_x^{(k^*)}|$ and the edge angle is θ_{k^*}. This completes the point filter.

4 Step 3: The Line Extension (Making It Better, But Slower)

The point filter works, but it only looks at one circular patch around the target pixel. For low-contrast edges or noisy images, one patch might not give a strong enough signal. The line filter fixes this by looking at multiple patches arranged along the candidate edge direction.

4.1 Virtual Evaluation Points

Instead of fitting the polynomial only at (X_0, Y_0), we also fit it at $(2m + 1)$ virtual positions spaced along the edge direction:

$$\left(X_j, Y_j\right) = (X_0 + j\cos\theta_k, Y_0 + j\sin\theta_k), \quad j \in \{-m, ..., 0, ..., m\} \tag{8}$$

For $m = 7$, this gives 15 positions (from $j = -7$ to $j = 7$). At each one, we run the entire point filter: gather N_p neighbors, build the matrix, solve, extract $\hat{f}_x^{(j,k)}$.

Think of it this way. We are sliding a circular patch along the candidate edge direction and measuring the edge strength at each position. If there really is an edge running in that direction, all 15 patches should see it, and the combined evidence is much stronger than any single patch.

4.2 Combining the Estimates

We combine the 15 derivative estimates using a Gaussian-weighted average:

$$R_k = \sum_{j=-m}^{m} w_j \cdot \hat{f}_x^{(j,k)} \tag{9}$$

where the weights are:

$$w_j = \exp\left(-\frac{j^2}{2\sigma^2}\right), \quad \sigma = \frac{m}{2} \tag{10}$$

The Gaussian weighting means positions near the center (small $|j|$) count more than positions far away (large $|j|$). This is sensible: the target pixel is at $j = 0$, and positions farther away are less relevant.

For $m = 7$, the weights look like this: $w_0 = 1.0$ (center), $w_{\pm 1} \approx 0.92$, $w_{\pm 3} \approx 0.57$, $w_{\pm 7} \approx 0.007$ (almost zero at the ends).

4.3 Why This Is Expensive

Every one of those 15 virtual positions requires its own gather of N_p pixels, its own matrix-vector product with the pseudoinverse, and its own extraction of the derivative. With $N_p = 100$ and $m = 7$:

- $15 \times 100 = 1500$ pixel reads per direction
- 15 separate matrix-vector multiplies
- All repeated for $N_s = 18$ directions

That is $15 \times 100 \times 18 = 27,000$ pixel reads per target pixel, plus thousands of multiply-add operations. For a 1280 x 720 image, that is about 25 billion pixel reads. This is why the naive LF is slow and uses tons of memory.

5 Step 4: The Algebraic Collapse (The Key Step)

Here is where the magic happens. We are going to show that the entire line filter computation, all 15 polynomial fits combined with Gaussian weighting, is equivalent to a single weighted sum over pixel values. No matrices, no pseudoinverses, no loops at runtime.

5.1 Expanding the Line Response

Let us write out what R_k actually computes, in full detail. From Equation 9 and Equation 6:

$$R_k = \sum_{j=-m}^{m} w_j \cdot \hat{f}_x^{(j,k)} \tag{11}$$

Substituting the definition of $\hat{f}_x^{(j,k)}$:

$$R_k = \sum_{j=-m}^{m} w_j \cdot \left(\boldsymbol{p}_{\text{fx}}^{(k)} \cdot \boldsymbol{b}_j\right) \tag{12}$$

where $\boldsymbol{b}_j$ is the vector of brightness values gathered around virtual position j. Now expand the dot product:

$$R_k = \sum_{j=-m}^{m} w_j \sum_{i=1}^{N_p} p_i^{(k)} \cdot f\big(X_j + \Delta x_i, Y_j + \Delta y_i\big) \tag{13}$$

where $p_i^{(k)}$ is the i-th entry of $\boldsymbol{p}_{\text{fx}}^{(k)}$ and $(\Delta x_i, \Delta y_i)$ are the offsets of the i-th neighbor relative to the center of its patch.

Now substitute the definition of $\big(X_j, Y_j\big)$ from Equation 8:

$$R_k = \sum_{j=-m}^{m} \sum_{i=1}^{N_p} \underbrace{w_j \cdot p_i^{(k)}}_{\text{a single number}} \cdot f\left(\underbrace{X_0 + j\cos\theta_k + \Delta x_i}_{\text{x-coordinate of pixel}}, \underbrace{Y_0 + j\sin\theta_k + \Delta y_i}_{\text{y-coordinate of pixel}} \right) \tag{14}$$

Let us pause and stare at this equation. It says: R_k is a sum of $(2m+1) \times N_p$ terms. Each term is a single number ($w_j \cdot p_i^{(k)}$, which we can precompute) multiplied by a single pixel value (read from the image at a specific location).

That is it. The entire line filter, the polynomial fitting, the pseudoinverse, the Gaussian weighting, all of it, reduces to "multiply some pixels by some precomputed numbers and add up." This is the algebraic collapse.

5.2 Why Does This Work?

It works because every operation in the pipeline is linear:

1. Gathering neighbor pixels is linear (just selecting values from the image).
2. The matrix-vector product $\boldsymbol{P}_{\theta_k}\boldsymbol{b}$ is linear in $\boldsymbol{b}$.
3. Extracting one row of the result is linear.
4. Multiplying by w_j is linear.
5. Summing over j is linear.

When you compose linear operations, the result is linear. A linear operation on pixel values is always expressible as "multiply each pixel by a weight and sum." There is no escaping this.

To make it even more concrete: imagine the entire image is a single vector $\boldsymbol{I}$ of all pixel values. Then R_k is some vector $\boldsymbol{\alpha}_k$ dotted with $\boldsymbol{I}$. The vector $\boldsymbol{\alpha}_k$ has a nonzero entry for every pixel that participates in the computation, and the value of that entry is the total weight that pixel receives. Most entries of $\boldsymbol{\alpha}_k$ are zero (pixels far from the target do not participate), so $\boldsymbol{\alpha}_k$ is sparse. But it is still just a dot product.

6 Step 5: Building the Fused Stencil

We have shown that R_k is a weighted sum over pixel values. But Equation 14 has $(2m+1) \times N_p$ terms. Many of those terms access the same pixel, because the circular neighborhoods of adjacent virtual positions overlap. We need to merge those duplicates.

6.1 The Overlap Problem (With Numbers)

Consider $m = 7$ and $N_p = 100$. Each of the 15 virtual positions gathers 100 neighbors from an approximately circular patch of radius ≈ 6 pixels. Adjacent virtual positions are only 1 pixel apart. So two adjacent patches share most of their pixels.

Concretely, virtual position $j = 0$ gathers the 100 pixels closest to (X_0, Y_0). Virtual position $j = 1$ (at $\theta_k = 0°$, this is one pixel to the right) gathers the 100 pixels closest to $(X_0 + 1, Y_0)$. These two circles of radius 6, centered 1 pixel apart, overlap in about 90 of their 100 pixels.

The raw stencil has $15 \times 100 = 1500$ entries, but many of those entries point to the same pixel.

6.2 Deduplication

We group all (j, i) pairs that map to the same integer pixel position and sum their weights:

$$\alpha_{k,\ell} = \sum_{\{(j,i):\ \mathrm{round}(j\cos\theta_k+\Delta x_i,\, j\sin\theta_k+\Delta y_i)=\ \mathrm{position}\ \ell\}} w_j \cdot p_i^{(k)} \tag{15}$$

After deduplication, the stencil shrinks dramatically. For $m = 7$, $N_p = 100$: from 1500 raw entries down to about 264 unique pixel positions. That is an 82% reduction.

The fused stencil for orientation k is then just two arrays, one giving the offsets $\left(\tilde{\delta}_\ell^x, \tilde{\delta}_\ell^y\right)$ and one giving the corresponding weight $\alpha_{k,\ell}$, for $\ell = 1, ..., N_k'$ where N_k' is the number of unique positions.

6.3 The Final Formula

After deduplication, the line filter response is:

$$R_k = \sum_{\ell=1}^{N_k'} \alpha_{k,\ell} \cdot f\left(X_0 + \tilde{\delta}_\ell^x, Y_0 + \tilde{\delta}_\ell^y\right) \tag{16}$$

Or in vector notation:

$$R_k = \boldsymbol{\alpha}_k^\top \boldsymbol{g}_k \tag{17}$$

where $\boldsymbol{\alpha}_k \in \mathbb{R}^{N_k'}$ is the precomputed weight vector and $\boldsymbol{g}_k \in \mathbb{R}^{N_k'}$ is the vector of pixel values gathered from the image at the stencil positions.

This is mathematically identical to Equation 14. The same pixels, the same total weights, the same answer. We have just reorganized the computation.

7 Step 6: Why This Is So Much Faster

7.1 The Naive LF (Before Fusion)

For each pixel, for each of N_s orientations, the naive LF does:

1. Loop over $(2m + 1)$ virtual positions.
2. At each virtual position, gather N_p neighbors from the image (a scattered memory read).
3. Compute $\boldsymbol{P}_{\theta_k} \boldsymbol{b}_j$ (a matrix-vector product: $M \times N_p$ multiplies).
4. Extract the normal derivative (pick one element).
5. Multiply by w_j.
6. Sum the $(2m + 1)$ weighted derivatives.

Total work per pixel per orientation: $(2m + 1) \times N_p$ pixel reads, plus $(2m + 1)$ matrix-vector products of size $M \times N_p$.

Total pixel reads per image pixel: $(2m + 1) \times N_p \times N_s$. For $m = 7, N_p = 100, N_s = 18$: $15 \times 100 \times 18 = 27,000$ reads. Plus storing all those intermediate values in GPU memory.

7.2 The Fused ASF (After Fusion)

For each pixel, for each of N_s orientations, the ASF does:

1. Gather N_k' pixel values from the image (one set of scattered reads).
2. Multiply each by its precomputed weight and sum (one dot product).

Total pixel reads per image pixel: $N_k' \times N_s$. For $m = 7, N_p = 100, N_s = 18$: $264 \times 18 = 4,752$ reads. That is 5.7 times fewer reads, and zero matrix operations.

But the speedup is even larger than 5.7 times because:

- No matrix-vector products at runtime (those are baked into the weights).

- No intermediate storage (the naive approach stores huge tensors; the ASF stores only the input image and the output).
- The GPU kernel is a simple loop with predictable memory access, which modern hardware loves.

In practice, we measured 18 to 24 times speedup and 311 times reduction in GPU memory.

8 Step 7: A Worked Example

Let us trace through the entire process with tiny numbers to make it concrete. We will use $d = 1$ (linear polynomial, 3 unknowns), $N_p = 4$ (4 neighbors), $m = 1$ (3 virtual positions), $N_s = 1$ (one orientation, $\theta = 0$, vertical edge).

8.1 The Point Filter Weights

With $d = 1$ and $\theta = 0°$, the polynomial model is $f(x, y) \approx f^0 + f^0_x x + f^0_y y$.

Suppose our 4 neighbors (relative to the center) are at offsets:

$$(\Delta x, \Delta y) \in \{(-1, 0), (1, 0), (0, -1), (0, 1)\} \tag{18}$$

The design matrix is:

$$\boldsymbol{A} = \begin{pmatrix} 1 & -1 & 0 \\ 1 & 1 & 0 \\ 1 & 0 & -1 \\ 1 & 0 & 1 \end{pmatrix} \tag{19}$$

The pseudoinverse (which you can verify by computing $\left(\boldsymbol{A}^\top \boldsymbol{A}\right)^{-1} \boldsymbol{A}^\top$) is:

$$\boldsymbol{P} = \begin{pmatrix} \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ -\frac{1}{2} & \frac{1}{2} & 0 & 0 \\ 0 & 0 & -\frac{1}{2} & \frac{1}{2} \end{pmatrix} \tag{20}$$

Row 1 (the one we care about) is $\boldsymbol{p}_{\text{fx}} = [-1/2, 1/2, 0, 0]$.

This makes perfect sense. To estimate the horizontal derivative, the polynomial fit says "take the right neighbor minus the left neighbor, divided by 2." The top and bottom neighbors get zero weight because they carry no information about the horizontal derivative. This is reassuringly intuitive.

8.2 The Line Extension

With $m = 1$, we have 3 virtual positions at $j \in \{-1, 0, 1\}$. Since $\theta = 0°$, these are at horizontal offsets $(-1, 0), (0, 0), (1, 0)$.

Gaussian weights with $\sigma = \frac{m}{2} = 0.5$:

$$w_{-1} = \exp\left(-\frac{1}{2 \cdot 0.25}\right) = \exp(-2) \approx 0.135 \tag{21}$$

$$w_0 = \exp(0) = 1.0 \tag{22}$$

$$w_1 = \exp(-2) \approx 0.135 \tag{23}$$

8.3 The Raw Stencil

For each virtual position j, the 4 neighbors (at offsets from that virtual position) map to these absolute offsets from the target pixel:

- $j = -1$ (virtual position at $(-1, 0)$): neighbors at $(-2, 0), (0, 0), (-1, -1), (-1, 1)$
- $j = 0$ (virtual position at $(0, 0)$): neighbors at $(-1, 0), (1, 0), (0, -1), (0, 1)$
- $j = 1$ (virtual position at $(1, 0)$): neighbors at $(0, 0), (2, 0), (1, -1), (1, 1)$

The raw weight for each entry is $w_j \cdot p_i$:

j	i	Offset	w_j	p_i	$w_j \cdot p_i$
-1	1	(-2, 0)	0.135	$-\frac{1}{2}$	-0.068
-1	2	(0, 0)	0.135	$\frac{1}{2}$	$+0.068$
-1	3	(-1, -1)	0.135	0	0
-1	4	(-1, 1)	0.135	0	0
0	1	(-1, 0)	1.0	$-\frac{1}{2}$	-0.500
0	2	(1, 0)	1.0	$\frac{1}{2}$	$+0.500$
0	3	(0, -1)	1.0	0	0
0	4	(0, 1)	1.0	0	0
1	1	(0, 0)	0.135	$-\frac{1}{2}$	-0.068
1	2	(2, 0)	0.135	$\frac{1}{2}$	$+0.068$
1	3	(1, -1)	0.135	0	0
1	4	(1, 1)	0.135	0	0

Table 1: Raw stencil entries. 12 total (3 virtual positions times 4 neighbors).

8.4 Deduplication

Some offsets appear more than once. Specifically, $(0, 0)$ appears at $(j = -1, i = 2)$ with weight $+0.068$ and at $(j = 1, i = 1)$ with weight -0.068. These sum to zero.

Also, many entries have weight zero (the neighbors with $p_i = 0$), so we can drop them.

After deduplication, the fused stencil is:

Offset	Fused weight α
(-2, 0)	-0.068
(-1, 0)	-0.500
(0, 0)	0.000 (drops out)
(1, 0)	$+0.500$
(2, 0)	$+0.068$

Table 2: Fused stencil after deduplication. Reduced from 12 entries to 4 nonzero entries.

The result: $R_0 = -0.068 \cdot f(-2, 0) - 0.500 \cdot f(-1, 0) + 0.500 \cdot f(1, 0) + 0.068 \cdot f(2, 0)$.

This is a weighted central difference that emphasizes the immediate neighbors (weight 0.5) and lightly includes the next-nearest neighbors (weight 0.068). It is like a slightly wider, smarter version of the Sobel filter, computed automatically from the polynomial fitting + line extension math. And we can compute this weighted sum directly without ever building a matrix or solving a system.

9 Step 8: The General Case

The worked example used tiny numbers. In the real filter with $N_p = 100$, $m = 7$, and $d = 4$, the process is identical but the numbers are bigger.

The raw stencil has $(2m + 1) \times N_p = 15 \times 100 = 1500$ entries. After deduplication to unique pixel positions, about 264 entries remain. Each entry has a precomputed weight $\alpha_{k,\ell}$ that encodes the combined effect of the polynomial fitting, the pseudoinverse extraction, and the Gaussian line weighting.

The stencil shape is an elongated, anisotropic pattern that stretches along the candidate edge direction. When you plot the weights, you see a dipole pattern: positive weights on one side of the expected edge, negative weights on the other. This makes sense because the derivative should respond to the difference in brightness across the edge.

At runtime, for each pixel, the GPU kernel loops over the N_s orientations, and for each one, it gathers the pixel values at the stencil offsets, multiplies by the precomputed weights, and sums. The orientation with the largest absolute response wins.

10 Summary

The derivation has four conceptual steps.

The first step is recognizing that the point filter (polynomial fit followed by derivative extraction) is a linear operation on pixel values. The derivative at a single point is just $\boldsymbol{p}_{\text{fx}}^{(k)} \cdot \boldsymbol{b}$, a dot product of fixed weights with brightness values.

The second step is recognizing that the line extension (evaluating the point filter at multiple virtual positions and taking a Gaussian-weighted sum) is also linear. You are summing dot products, which is itself a dot product.

The third step is expanding the double sum (over virtual positions j and neighbors i) to get a single sum over (j, i) pairs, each pair contributing $w_j \cdot p_i^{(k)}$ times one pixel value.

The fourth step is deduplication: grouping all (j, i) pairs that reference the same pixel, summing their weights, and storing only the unique positions and fused weights. This compressed stencil is the ASF.

The result is mathematically identical to the original line filter. Same pixels, same weights, same answer. But the computation goes from "build matrices, solve systems, loop over virtual positions" to "gather pixels, multiply by precomputed weights, sum." This is why the ASF is 18 to 24 times faster and uses 311 times less memory than the naive LF implementation.